

第五章

面向对象分析



1 面向对象方法学

1.1 结构化开发方法的不足

结构化开发方法的核心思想：

结构化方法的核心思想是将整个软件按功能分解为一个个处理模块，每个模块完成一个基本功能，多个模块组合在一起实现整个系统的功能。由于软件设计的问题，一些模块之间的耦合可能比较紧密，这样就会出现一个模块的修改对系统其他部分的影响比较大。

结构化开发方法的不足：

- 由于开发人员对用户需求的理解存在偏差，开发完成的软件与用户的实际需求常常有差别，不能满足用户的需要。
- 软件开发效率低下。用结构化方法开发软件时受需求变化的影响非常大，导致软件开发过程中的不断返工。
- 软件维护困难。一个模块的修改对系统其他部分的影响比较大，使得维护系统比较困难。
- 软件可复用性差。结构化方法开发的软件除了一些接口简单的数学函数外，其他软件的可复用性一般都比较差。

1.2 面向对象开发方法

- 面向对象方法是一种把面向对象的思想应用于软件开发过程中，指导开发活动的系统方法，简称OO (Object-Oriented)方法，是建立在“对象”概念基础上的方法学。
- 面向对象方法学的出发点和基本原则，是尽可能模拟人类习惯的思维方式，使开发软件的方法与过程尽可能接近人类认识世界、解决问题的过程。
- 通常，客观世界中的实体有静态属性又有动态行为，然而传统语言提供的解空间对象实质上仅仅是描述实体属性的数据，必须在程序中从外部对其施加操作，才能模拟它的行为。而实际上，数据以及对数据施加的处理是密不可分的。
- 面向对象方法将数据与对数据的处理封装在一个统一体中，外界只需要发消息请求对象执行它的某些操作。

面向对象方法学

面向对象分析和设计解决了两个经典问题

- 问题一：传统的分析和设计方法将数据模型和处理模型分离。处理模型和数据模型实际上是描述解决一个问题的两个方面，但是建立两个模型使得系统分析人员很难检查它们的一致性和正确性。
- 问题二：如何从系统分析平滑地过渡到系统设计？传统的分析方法中分析模型到设计模型转变并不规范，容易引入错误。

UML将分析和设计模型统一，所使用的符号统一。

1.3面向对象常用概念

- 对象
- 类
- 消息
- 方法
- 继承多态性
- 重载

2 UML统一建模语言

2.1 面向对象分析方法



- 上世纪90年代，提出来很多种面向对象设计方法，每种方法都有一套自己的描述符号和实现过程，每种方法都支持3种基本的活动：识别对象和类，描述对象和类之间的关系，通过描述每个类的功能定义对象的行为。
- Booch方法
- OMT方法
- Coad/Yourdon方法
- OOSE方法等

2.2 UML简介

- 1996年发布，UML (Unified Modeling Language) 是软件界第一个统一的建模语言，该方法结合了Booch，OMT，和OOSE方法的优点，统一了符号体系，并从其它的方法和工程实践中吸收了许多经过实际检验的概念和技术。它是一种标准的表示，为不同领域的用户提供了统一的交流标准 – UML图。
- UML的价值在于它综合并体现了世界上面向对象方法的最好实践经验，支持用例驱动，以架构为中心，递增和迭代地进行软件开发。

UML的类型

特点

- UML的特点：统一标准、面向对象、可视化描述能力强、 独立于过程、易掌握、易用
- UML的重要内容可以由下列五类图（共10种图形）来定义：
- 第一类是用例图。
- 第二类是静态图 (Static diagram)，包括类图、对象图和包图。
- 第三类是行为图 (Behavior diagram)，包括活动图和状态图。
- 第四类是交互图 (Interactive diagram)，包括顺序图和合作图。
- 第五类是实现图 (Implementation diagram)，包括组件图和部署图。

3 用例建模

3.1 用例与执行者



- 1992年由Jacobson提出了Use case 的概念及可视化的表示方法—Use case图，并加入由他提出的面向对象的软件工程(OOSE)。
- 用例建模技术，是从用户的角度来描述系统的功能需求，在宏观上给出模型的总体轮廓。通过对典型用例的分析，使开发者能够与用户进行充分交流，有效地了解和获取用户需求。
- Use case 的概念受到了IT界的欢迎，被广泛应用到了面向对象的系统分析中。基于用例的需求方法，已成为面向对象的分析方法的主流。
- 用例模型被推荐为获取和识别需求的首选工具!!

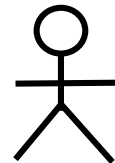
Use case图

- 采用“基于用例的方法”来识别和获取需求，是从外部的角度来看系统功能，建立系统的用例模型描述外部执行者所理解的系统功能，即待开发系统的功能需求。
- 用例模型由若干个用例图构成，在UML中构成用例图的主要元素是用例和执行者及它们之间的联系。
- **用例**-----表示一个子系统，或系统一个独立的功能。
- **角色**-----表示外部的“执行者”。

• **用例：**



角色：



连接：



(1) 用例

- 用例是系统提供的功能块，它揭示了人们如何使用系统。从本质上讲，一个用例是用户与计算机之间的一次典型交互作用。在UML中，用例被定义成系统执行的一系列动作（功能）。
- 在结构化的方法中，我们是将功能逐步分解成系统能够处理的模块。而在面向对象的方法中，我们关心的是用户要系统完成什么，用户要告诉系统什么信息，系统要返回给用户什么信息。
- 用例有以下特点：
 - 用例捕获某些用户可见的需求，实现一个具体的用户目标。
 - 用例由执行者激活，并提供确切的值给执行者。
 - 用例可大可小，但它必须是对一个具体的用户目标实现的完整描述。

用例

- 自动售货机系统的用例图，自动售货机系统由三个主要的用例组成：买饮料、供货、取货款（图中的自动售货机图片仅为增加直观性）



如何确定用例

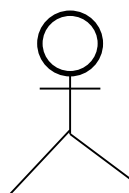
- 与系统实现有关的主要问题是什么？
- 系统需要哪些输入/输出？这些输入/输出从何来？到哪里去？
- 执行者需要系统提供哪些功能？
- 执行者是否需要对系统的信息进行读、创建、修改、删除或存储？
- 每个用例应该有一个相关的说明，描述该用例的作用。例如，自动售货机系统的售货用例说明：

售货——顾客投入钱币，选择所购商品，自动售货机系统检查该商品的数量及价格，并找零。

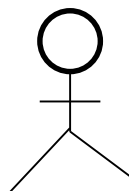
(2) 执行者(Actor)

- 执行者/参与者是指与系统交互的人或物。执行者在用例图中是用类似人的图形表示，但执行者可以是人，也可以是一个外界系统。
- 执行者触发用例，并与用例进行信息交换。一个执行者可与多个用例联系，一个用例也可与多个执行者联系。**注意：用例总是由执行者启动的**
- 执行者有3类：系统的使用者（即用户）、外部系统、时间（时钟）。

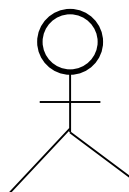
自动售货系统中的执行者



客户



供货人



收银员



图 自动售货系统

确定执行者(Actor)

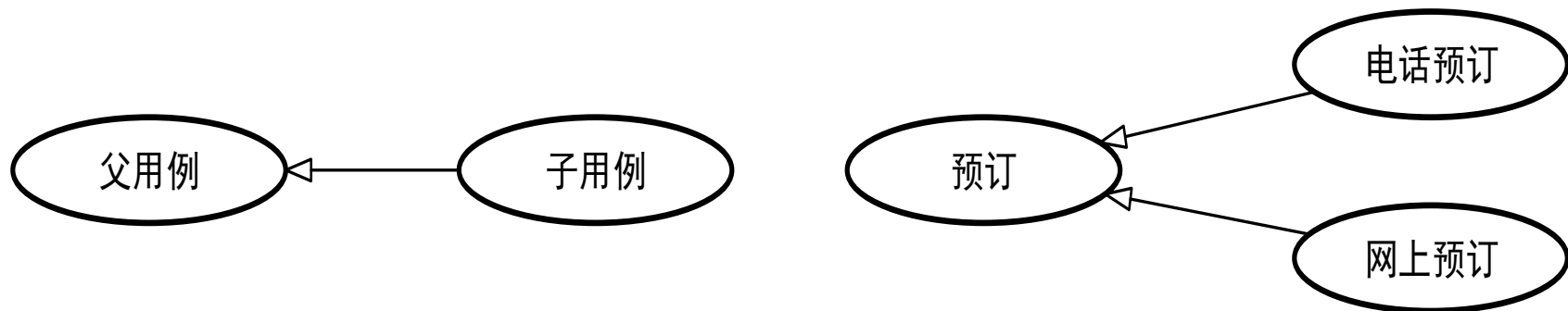
- 面对一个大系统，要列出所有用例的清单常常十分网难。比较可行的方法是先列出所有参与者的清单，再对每个参与者列出用例，问题就会变得比较容易。
- 如何确定执行者：
 1. 谁使用系统的主要功能(主执行者)?
 2. 谁需要从系统获得对日常工作的支持和服务?
 3. 谁需要维护管理系统的日常运行(副执行者)?
 4. 系统需要控制哪些硬件设备?
 5. 系统需要与其他哪些系统交互?
 6. 谁需要使用系统产生的结果?

3.2 用例关系

- 在用例图中除了执行者与用例之外，还必须描述用例之间的关系。用例之间存在以下几种关系。

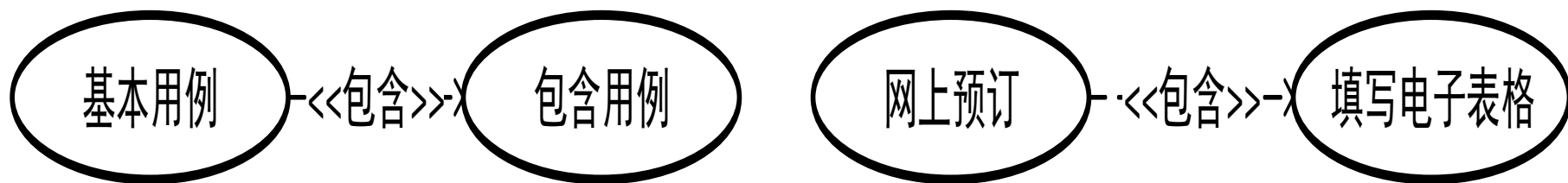
(1) 泛化关系(Generalization)代表一般与特殊的关系，类似继承。在用例泛化中，**子用例表示父用例的特殊形式，子用例继承了父用例的行为和属性**，也可以增加新的行为和属性或覆盖父用例中的行为。

- 例子：一个租赁或销售系统用例



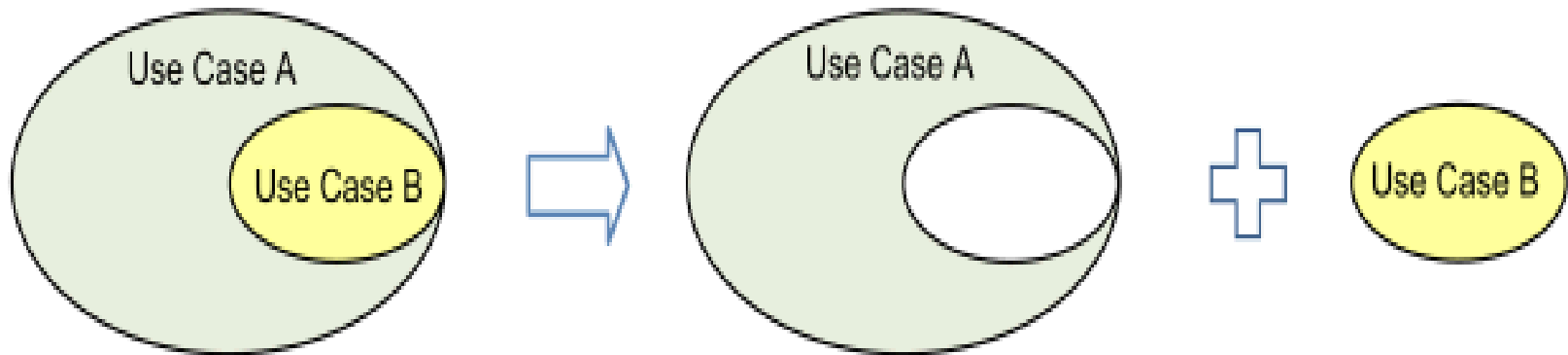
(2) 包含关系

- 一个用例（基用例，基本用例）可以包含其他用例（包含用例）具有的行为，并把它所包含的用例行为作为自身用例的一部分，这被称为包含关系。
- UML中，包含关系表示为虚线箭头加<<include>>箭头从基本用例指向包含用例。
- 例子：一个租赁或销售系统用例

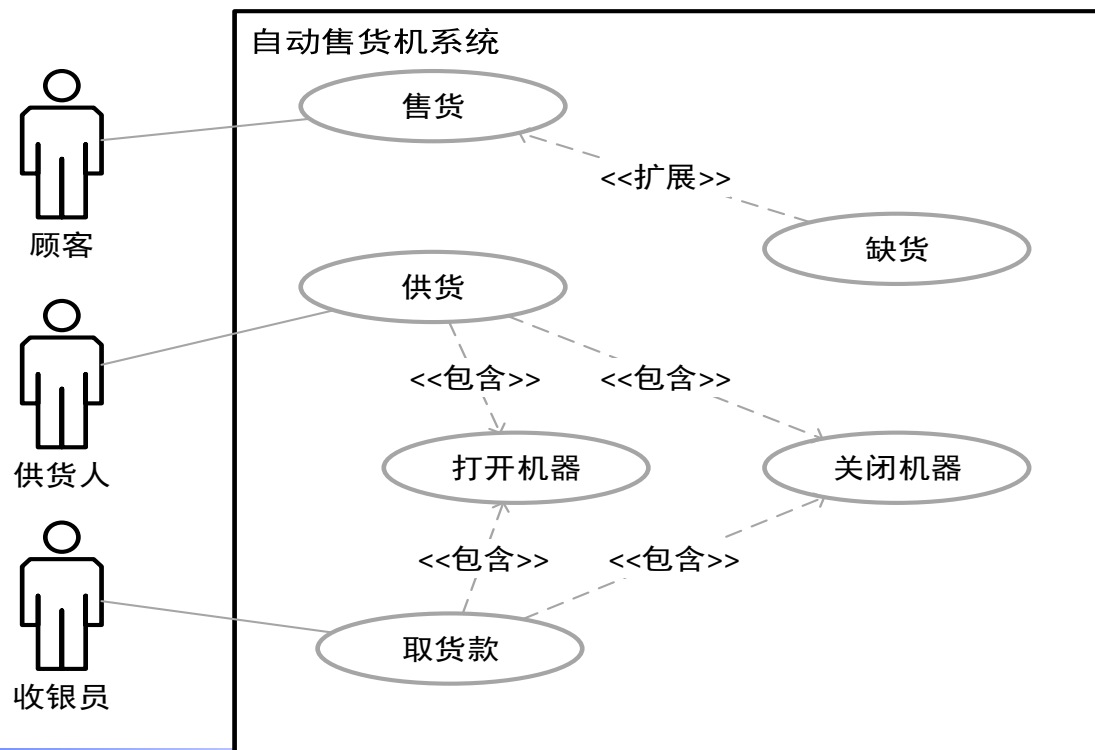


(2) 包含关系

- 包含关系的典型应用就是复用。下图显示从一个较大的用例A中抽取一部分较小的功能作为一个较小的用例B，则较大用例剩余部分构成的用例(A-B)在工作时要使用较小的用例B。将用例B从用例A中提取出来作为一个单独用例的作用是该用例B可以被多个其他用例所使用。

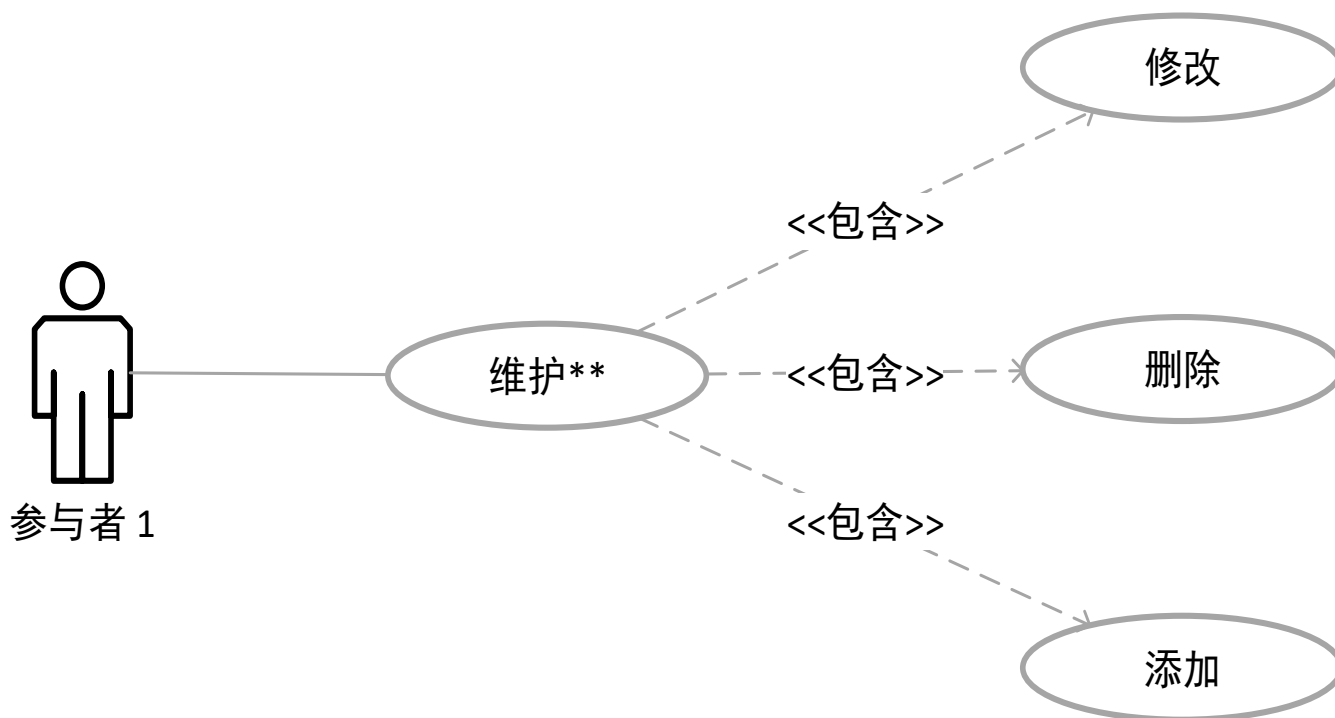


- 案例：自动售货机系统中供货与取货款这两个用例的开始动作都是打开机器，而它们最后的动作都是关闭机器。因此，将开始动作抽象为“打开机器”用例，将最后的动作抽象为“关闭机器”用例，则供货用例与取货款用例在执行时必须包含这两个用例



用例之间的关系-包含关系

- 有时当某用例的事件流过于复杂时，为了简化用例的描述，我们也可以把某一段事件流抽象成为一个被包含的用例。



(3) 扩展关系

- 扩展是将基用例中一段相对独立并且可选的动作，用扩展用例加以封装，再让它从基用例中声明的扩展点 (extension points) 上进行扩展，从而使基用例行为更简练、目标更集中。
- 箭头指向的用例为被拓展的用例，称为基础用例；箭头出发的用例为扩展用例。

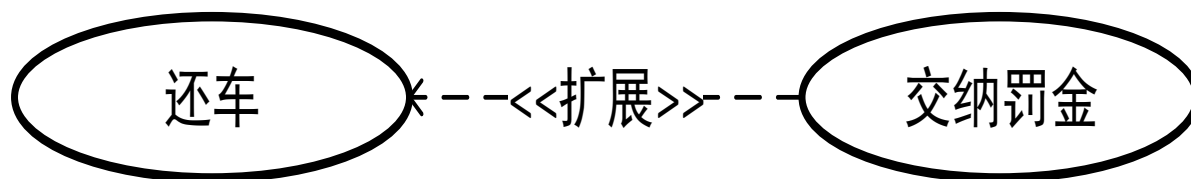


用例之间的关系-扩展关系

- 扩展关系可以有控制条件，当用例实例执行到达一个扩展点时，控制条件决定是否执行扩展。因此扩展关系可处理事件流的异常或者可选事件。
- 基本用例不知道扩展的任何细节。没有扩展用例，基本用例是完整的。
- 扩展关系与包含关系的区别是：没有扩展用例，基本用例是完整的。

扩展关系举例

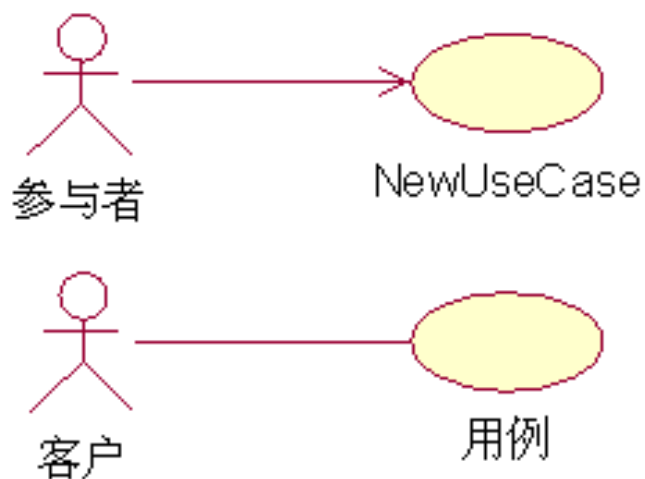
- **例：**汽车租赁系统用例图（部分）。基本用例是“还车”扩展用例是“交纳罚金”。如果一切顺利汽车可以被归还，那么执行“还车”用例即可。但是如果超过了还车的时间或汽车受损，按照规定客户要交纳一定的罚金，这时就不能执行提供的常规动作。因此可以在用例“还车”中增加扩展点，即特定条件为超时或损坏，如果满足条件，将执行扩展用例“交纳罚金”。



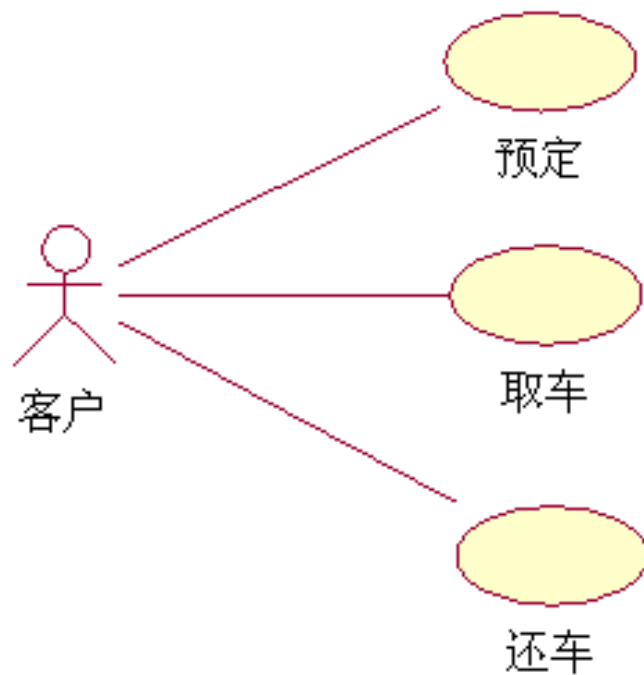
(4) 执行者与用例关系-关联关系

- 关联关系表示参与者和用例之间的通信。在UML中关联关系用直线或箭头表示。如果参与者启动用例，箭头指向用例；如果参与者利用了用例提供的服务，箭头指向参与者。如果二者是互动的，则是直线。
- **例：**汽车租赁系统用例图(部分)显示的是“客户”参与者以及与他交互的3个用例：预定、取车、还车。

参与者与用例的关系-关联关系



用例间的关联关系



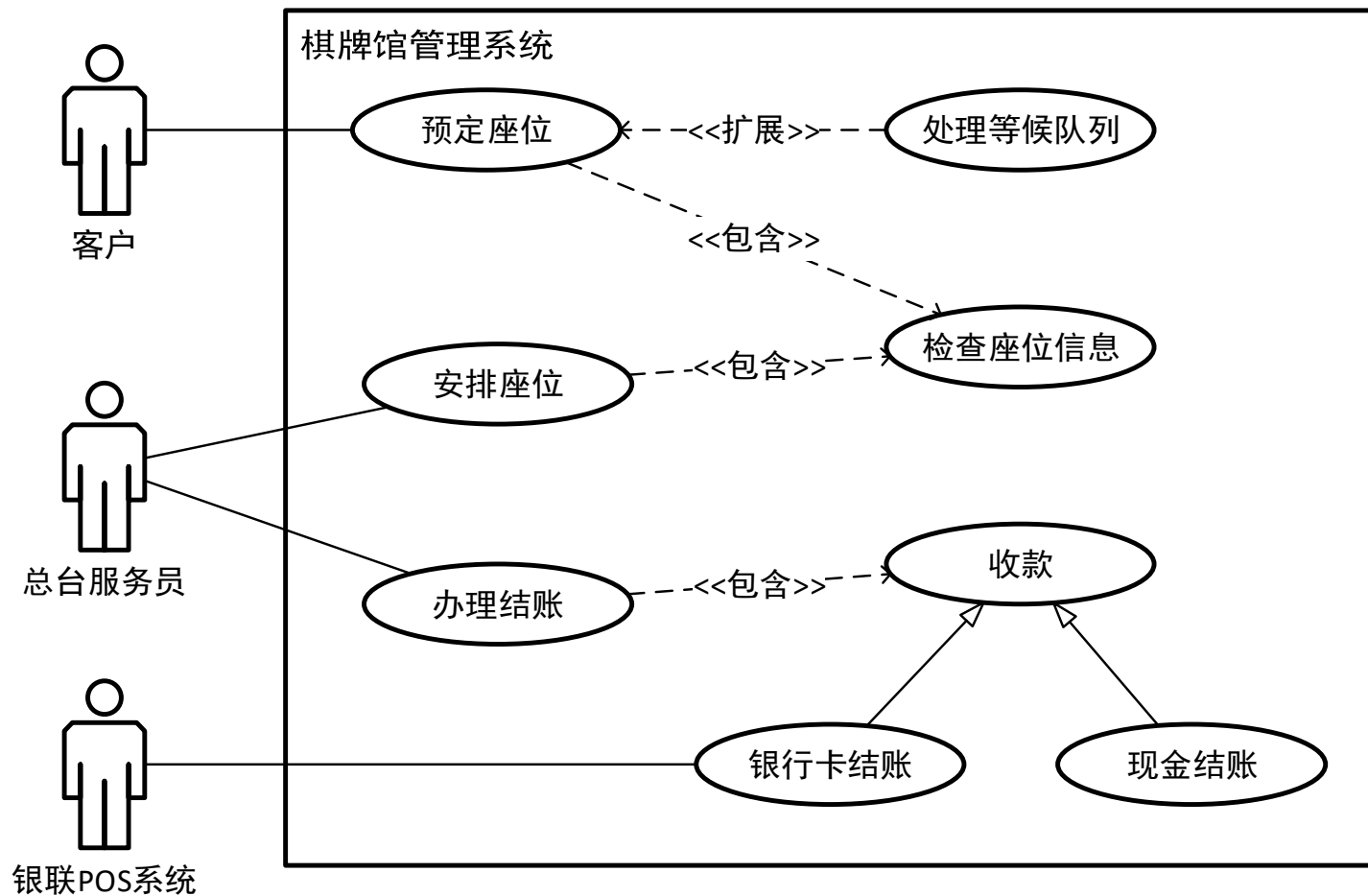
关联关系示例

案例

- **例子：棋牌馆管理系统用例模型局部**

系统主要功能：以Internet的形式向客户提供座位预定的服务，并且如果暂时无法获取座位的信息，允许客户进入“等候队列”，当有人退订之后即使通知客户。另外，该系统还将为总台服务员提供作座位安排以及结账的功能，要求能够支持现金和银行卡两种结账方式。

用例图

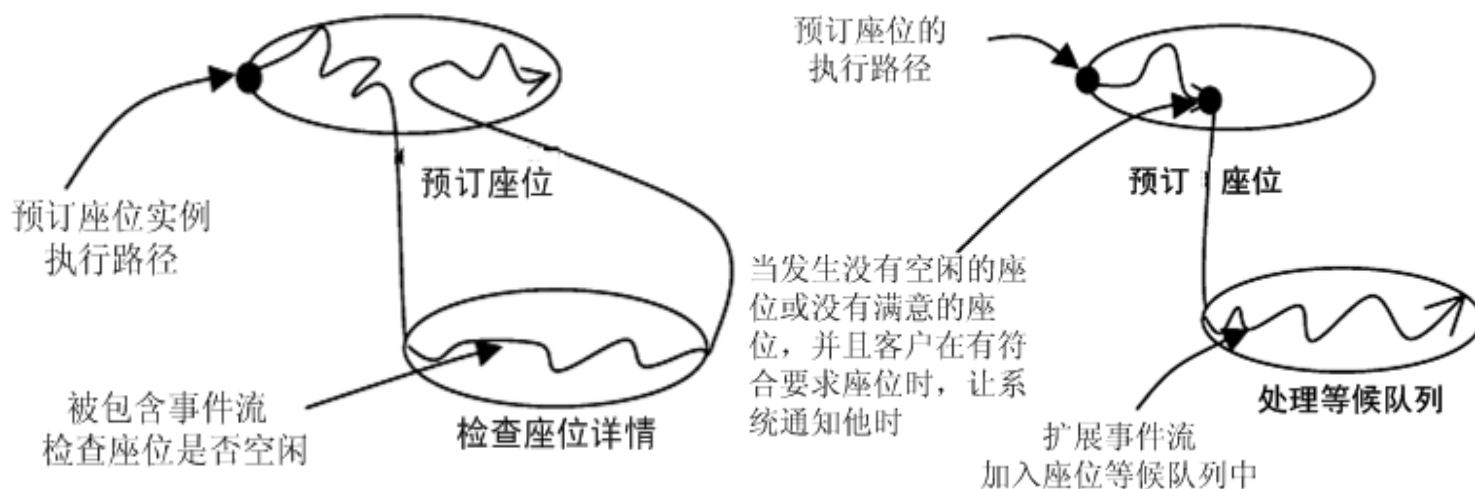


用例图—读图小结

- 首先定义了三个基用例：预订座位、安排座位和处理结账
- 客户通过Internet启动“预订座位”用例，在“预订座位”用例的执行过程中，将“检查座位信息”（被包含用例），如果没有空闲的座位或满意的座位，可以选择进入等候队列，这样就将启动扩展用例“处理等候队列”。
- 总台服务员在客户到棋牌馆时，启动“安排座位”，在执行过程中，启动被包含用例“检查座位信息”。
- 当客户要离开棋牌馆时，总台服务员将启动“处理结账”用例并且定义了两种“收款”用例，一个是“处理现金结账”，另一个是“处理银行卡结账”，而后一个用例将通过与外部系统“银联POS系统”交互来完成。（泛化关系）

该用例图中的关系

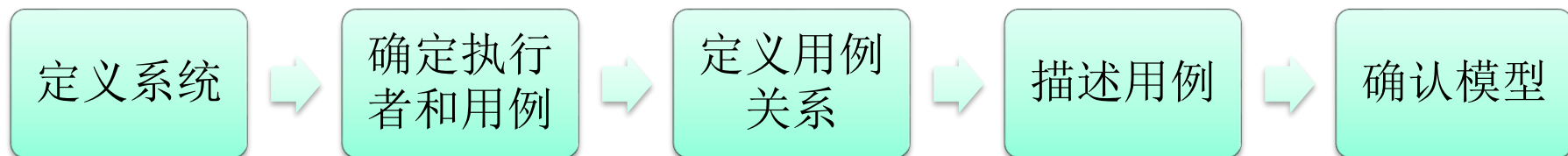
- **包含关系：**用例预定座位就包含了用例检查座位信息。被包含的用例（此例中的检查座位详情）不是孤立存在的，它仅作为某些包含它的更大的基用例（此例中的预订座位、安排座位）的一部分出现



- **扩展关系：**用例处理等候队列就是对用例预定座位的一个扩展。处理等候队列并不是在每次预定座位的时候都会发生。基用例是可以独立于扩展用例存在的，只是在特定的条件下，它的行为可以被另一个用例的行为所扩展

3.3 用例模型

- 建立系统用例模型的过程就是对系统进行功能需求分析的过程，用例建模的过程：



用例描述

项目	内容描述
ID	用例的标识
名称	对用例内容的精确描述，体现了用例所描述的任务
参与者	描述系统的参与者和每个参与者的目标
触发条件	标识启动用例的事件，可能是系统外部的事件，也可能是系统内部的事件，还可能是正常流程的第一个步骤
前置条件	用例能够正常启动和工作的系统状态条件
后置条件	用例执行完成后的系统状态条件
正常流程	在常见和符合预期的条件下，系统与外界的行为交互序列
扩展流程	用例中可能发生的其他场景
特殊需求	和用例相关的其他特殊需求，尤其是非功能性需求

案例：医院病房监护系统

根据分析系统主要实现以下功能：

- 1、病症监视器可以将采集到的病症信号（组合），格式化后实时的传送到中央监护系统。
- 2、中央监护系统将病人的病症信号开解后与标准的病症信号库里的病症信号的正常值进行比较，当病症出现异常时系统自动报警。
- 3、当病症信号异常时，系统自动更新病历并打印病情报告。
- 4、值班护士可以查看病情报告并进行打印。
- 5、医生可以查看病情报告，要求打印病情报告，也可以查看或要求打印病历。
- 6、系统定期自动更新病历。

例子：医院病房监护系统

- 用UML描述系统的静态结构
- 通过以下六个问题识别角色
 1. 谁使用系统的主要功能(主执行者)?
 2. 谁需要从系统获得对日常工作的支持和服务?
 3. 谁需要维护管理系统的日常运行(副执行者)?
 4. 系统需要控制哪些硬件设备?
 5. 系统需要与其他哪些系统交互?
 6. 谁需要使用系统产生的结果?

例子：医院病房监护系统

- 通过回答这六个问题以后，再进一步分析可以识别出本系统的四个角色：值班护士，医生，病人，标准病症信号库。
- 角色描述版

角色：病人
角色职责：
提供病症信号

角色职责识别：

负责生成、实时提供各种病症信号。

角色：医生

角色职责：
对病人负责，负责
处理病情的变化

角色职责识别：
(1)需要系统支持以完成其日常工作
(2)对系统运行结果感兴趣

角色：值班护士

角色职责：
负责监视病人的病情变化

角色职责识别：
(1)使用系统主要功能
(2)对系统运行结果感兴趣

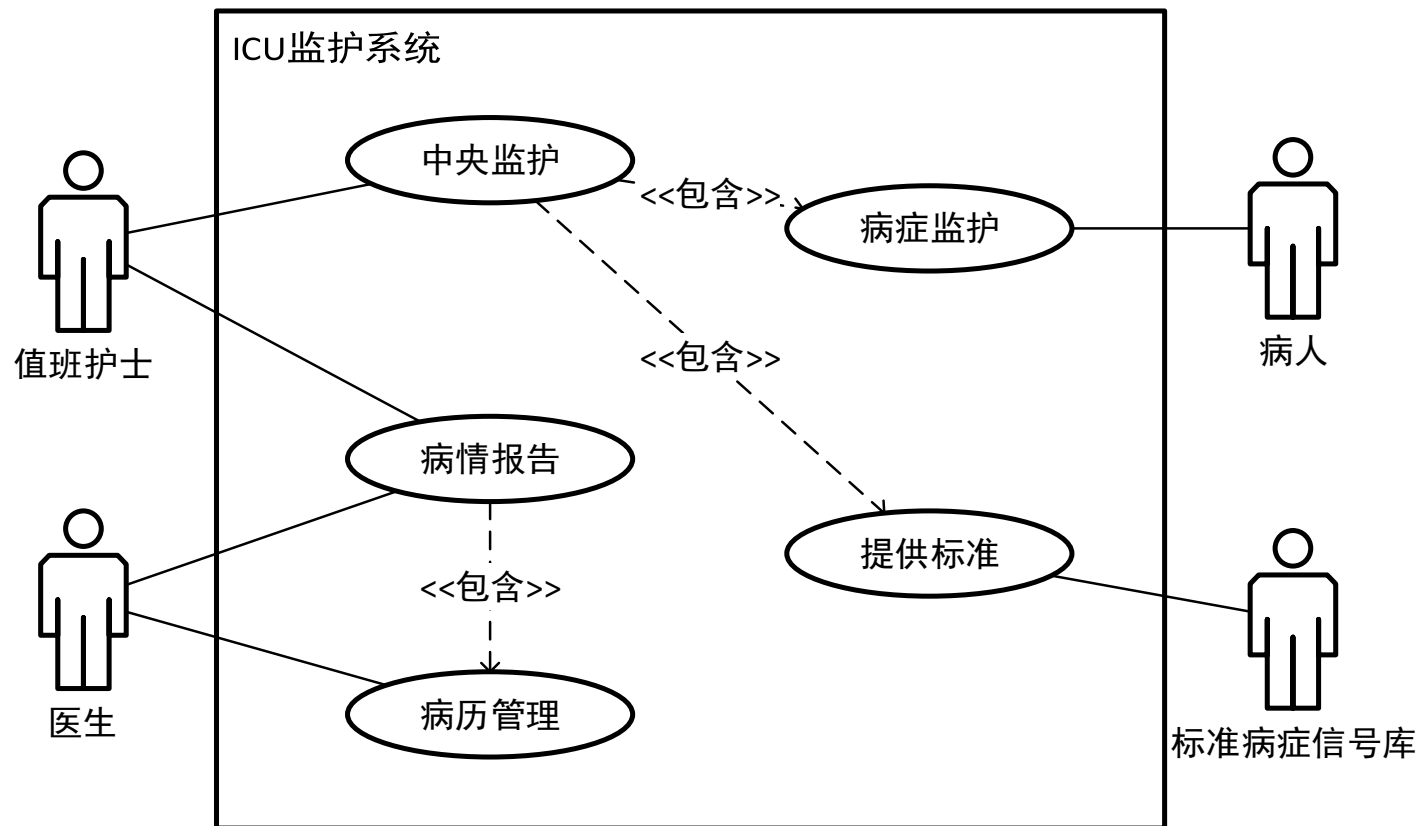
角色：标准病症信号库

角色职责：
负责向系统提供病症信号的正常值

角色职责识别：
(1)负责保持系统正常运行
(2)与系统交互

案例：医院病房监护系统

- 通过分析可以初步识别出系统的用例为：中央监护，病症监护，提供标准病症信号，病历管理，病情报告管理。顶层用例图



例子：医院病房监护系统

- 将用例细化，可以得到分解的用例：

1. 中央监护

分解为：a 分解信号 将从病症监护器传送来的组合病症信号分解为系统可以处理的信号。

b 比较信号 将病人的病症信号与标准信号比较。

c 报警 如果病症信号发生异常(即高于峰值)，发出报警信号。

d 数据格式化 将处理后的数据格式化以便写入病历库。

2. 病症监护

分解为：e 信号采集 采集病人的病症信号。

f 模数转化 将采集来的模拟信号转化为数字信号。

组信 g 信号数据组合 将采集到的脉搏，血压等信号数据组合为一号数据。

h 采样频率改变 根据病人的情况改变监视器采样频率。

3. 提供标准病症信号 i (此用例不分解)

例子：医院病房监护系统

4. 病历管理

分解为：j 生成病历

k 查看病历

l 更新病历

m 打印病历

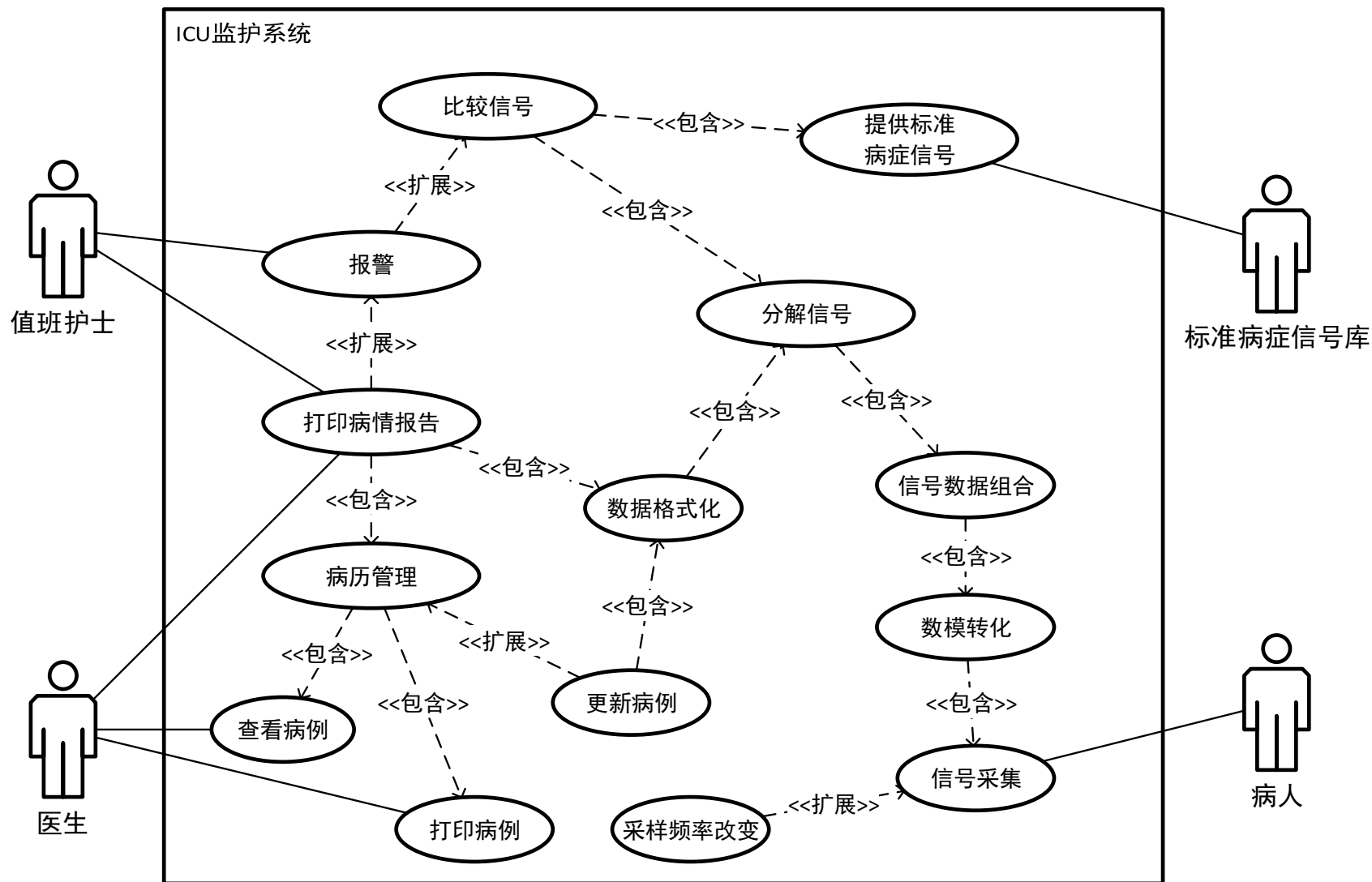
5. 病情报告

分解为：n 显示病情报告 在显示器上显示病情

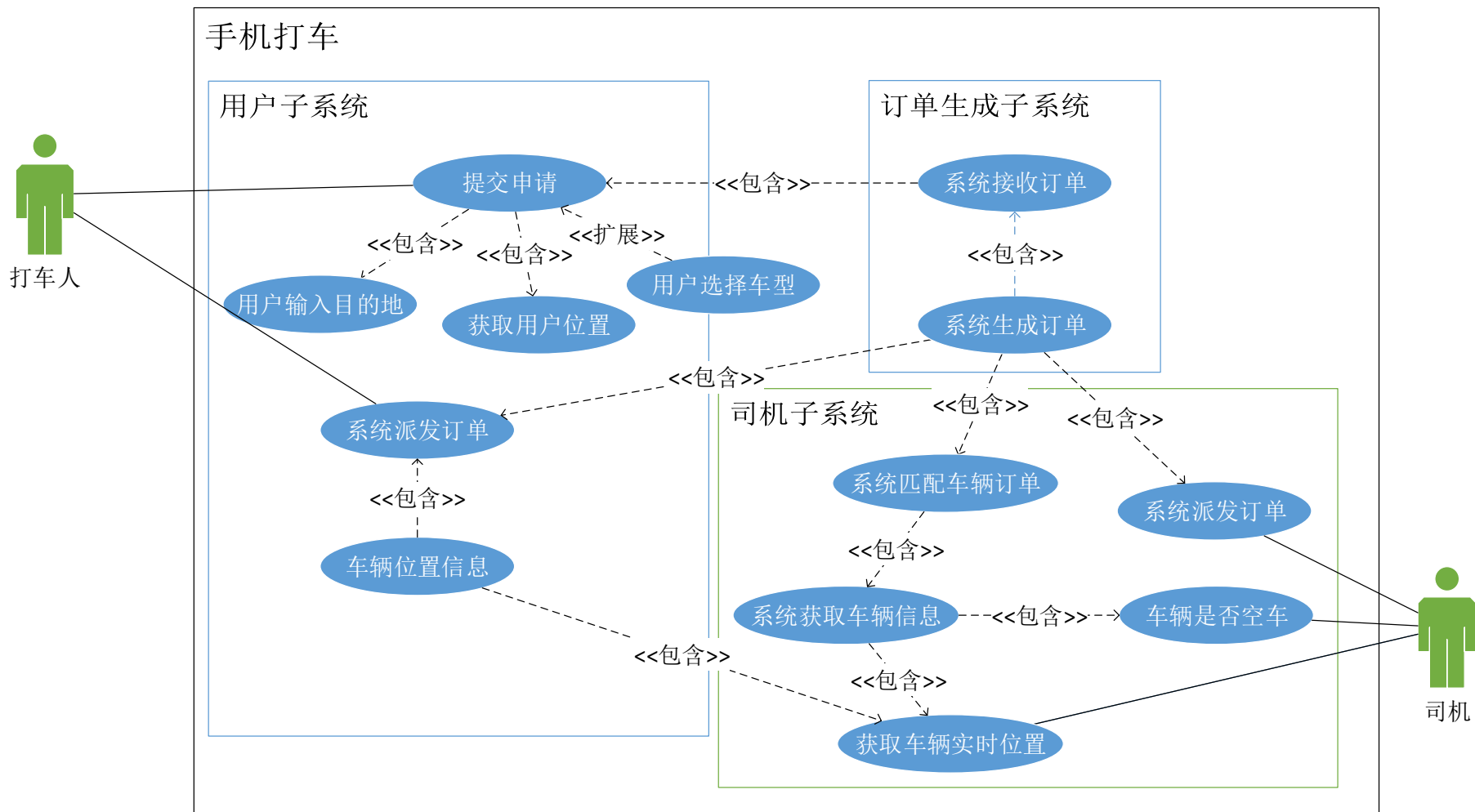
o 打印病情报告 在打印机打印病情报告

练习：画出用例图

例子：医院病房监护系统细化的用例图



案例：手机打车软件中发起打车申请的用户例图



4 活动图

4.1 活动图概述

- 活动图是一种用于描述系统行为的模型视图，它可用来描述过程（业务过程、工作流、事件流等）中的活动及其迁移。简单地讲，活动图是“面向对象的流程图”。
- 活动图能够附加在如下建模元素中以描述该元素的行为：
 - 用例
 - 类
 - 接口
 - 组件
 - 节点

活动图的特点

- (1) 活动图可以有效地描述整个系统的流程，即活动图描述的是系统的全局的动态行为。
- (2) 系统任务中存在大量的并发活动，只有活动图是唯一能够描述并发活动的UML图。
- (3) 活动图还描述了系统中各种活动的执行的顺序，刻画一个方法中所要进行的各项活动的执行流程。
- (4) 活动图能够充分刻画不同参与者之间的交互行为。

活动图的主要应用

- 1. 描述用例的行为

活动图对用例描述尤其有用，它可建模用例的工作流，显示用例内部和用例之间的路径；它也可以向读者说明需要满足什么条件用例才会有效，以及用例完成后系统保留的条件或者状态。

- 2. 理解工作流程

活动图对理解业务处理过程十分有用。可以画出描述业务工作流的活动图与领域专家进行交流，明确业务处理操作是如何进行的将会有怎样的变化。

- 3. 描述复杂过程的算法

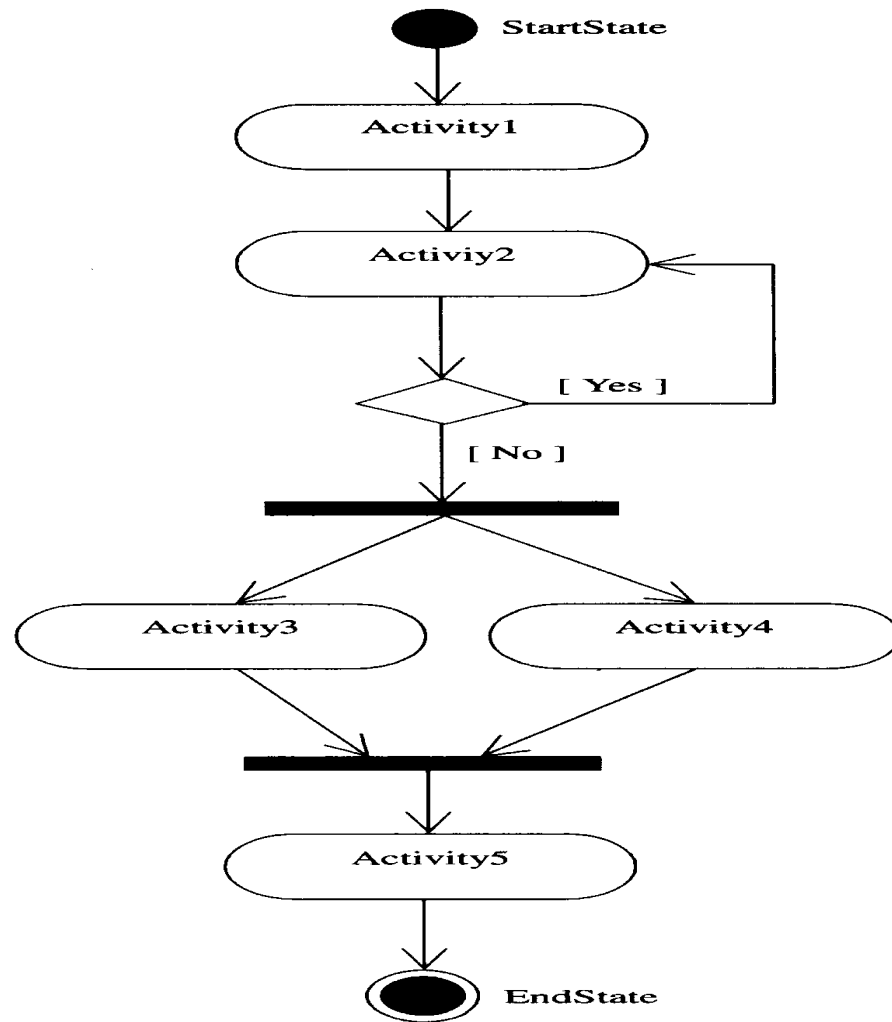
在这种情况下使用的活动图不过是UML版的程序流程图，常规的顺序、分支过程在活动图中都能得到充分的表现。

4.2 活动图元素

- 活动图保留了许多传统的流程图

的特征，它包含如下元素：

- ✓ 活动
- ✓ 操作
- ✓ 状态
- ✓ 转移
- ✓ 决策
- ✓ 分岔
- ✓ 联结
- ✓ 泳道



基本元素--活动

- 活动在活动图中，用来指示要完成某项工作的动作或表示工作流的步骤，具有内部动作的状态。其UML标记符是一个带有圆角的矩形，如下图所示。



- 在确定活动名称时应该恰当的命名，选择准确描述所发生动作的几个词。例如，Save File或者 Create New Document 就是比较恰当的活动名称，而Run或者Update 对读者而言是不完整的名称。

基本元素--操作

- 可以用操作在活动中增加更多详细的步骤。操作是活动中执行的小步骤。在下列情况下发生：
 - 进入活动时发生的操作，标有entry字样。
 - 活动进行时发生的操作，直到离开活动，标有do字样。
 - 离开活动时发生的操作，标有exit字样。
 - 特定事件发生时的操作，标有event字样和事件名。
- 操作是可选的，但提供的详细信息有助于后面完成系统设计。如果包括操作，则其在活动内显示，不管其属于上面哪一类。下面是带操作的活动例子。

显示可选航班

entry/寻找所选城市/日期的所有航班

entry/确定有空位的航班

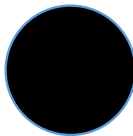
do/显示有空位的航班清单

do/加亮显示机票最便宜的航班

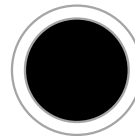
event/用户请求机票价信息/显示机票价信息

基本元素--状态

- 状态的标记符与活动类似，也是带圆角的矩形，但状态的圆角比较小。
- 状态通常使用一个指示系统当前状态的单词或者短语来标识。例如，Stopped是一个状态，而stop则是一个活动。



开始状态



结束状态

UML包含两个特殊状态，即开始状态和结束状态。开始状态以实心黑点表示，结束状态以带有圆圈的实心黑点表示。

- **注意：** 每一个活动图只能有一个开始状态，但是可以有无数个结束状态。如下图所示：

 Ready to record grades

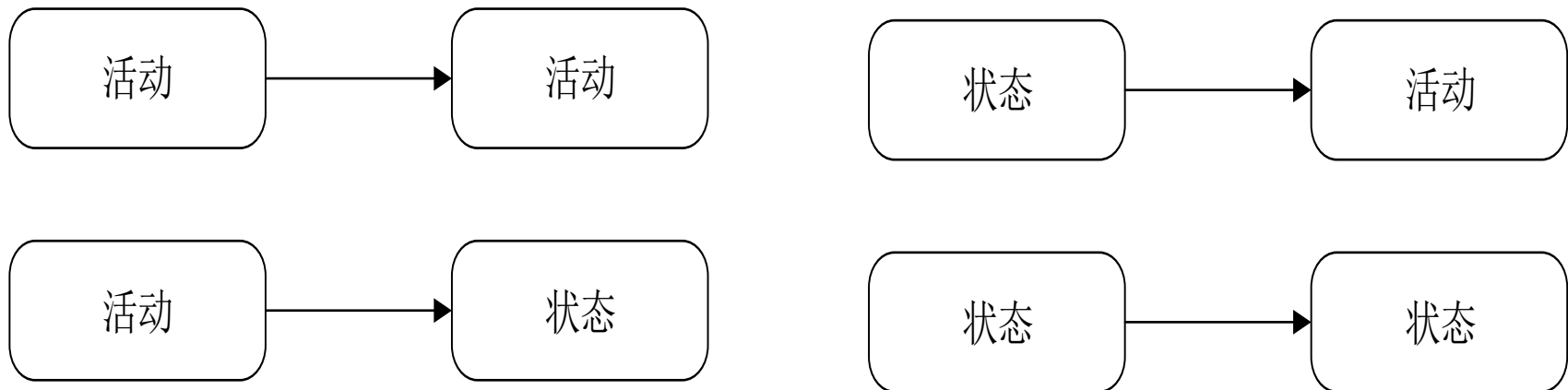
 Grades Recorded

 User Error

 Operation Cancelled

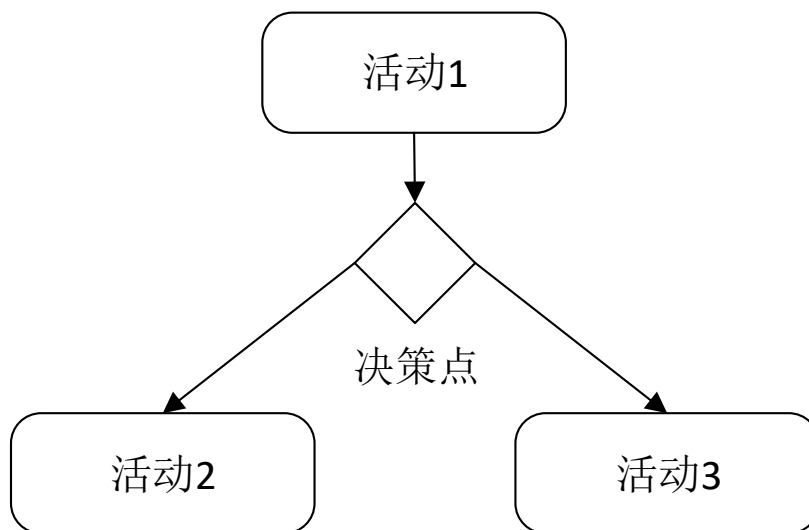
基本元素--转移

- 转移描述由于隐含事件引起的活动变迁，即转移可以连接各活动及特殊活动(初态、终态、决策、同步线)
- 转移用来指示一种状态到另一个状态的控制流，也可以显示活动之间或者状态之间的控制流。转移的标记符是带开放箭头的实线，如下图所示。



基本元素--决策

- **决策**是基于判断条件选择控制流继续的方向。决策的UML符号是一个小菱形标记符，然后从这里再按条件控制分支转移到满足条件的活动，如下图所示。

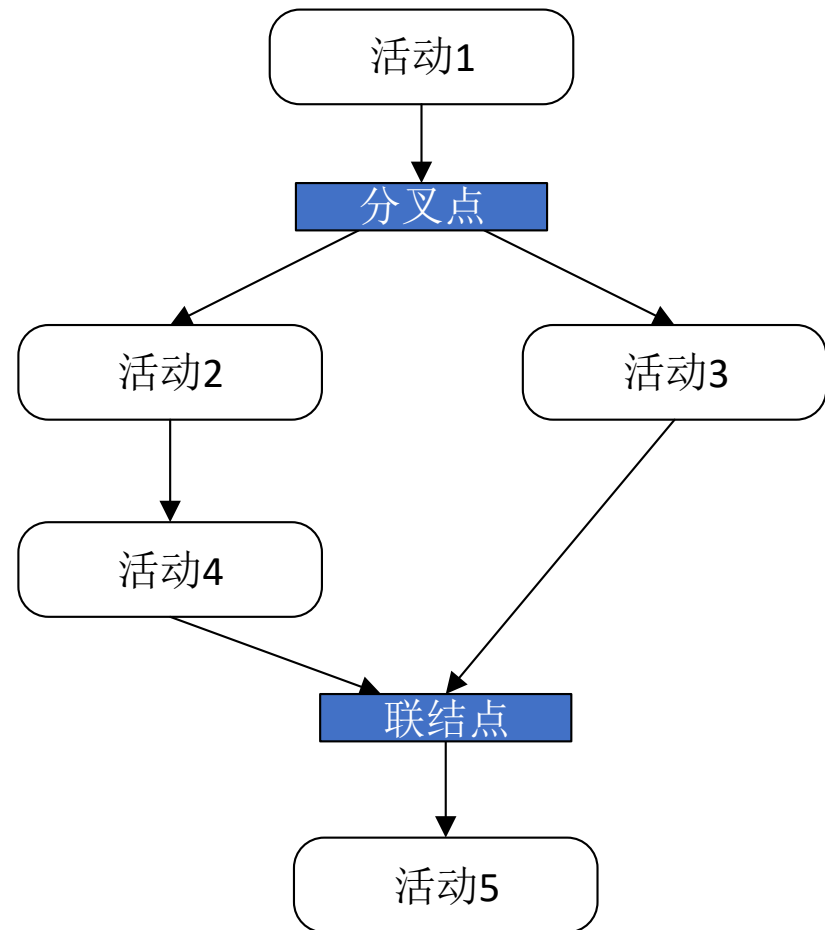


基本元素-分岔和联结

- **分岔**用来表示并行活动的分支处理，**联结**用来把并行活动汇集到同步处理。
- 分岔和联结在UML中的表示方法相似，都用粗黑线表示。分岔具有一个转移入口，两个或者多个转移出口。分岔描述了单向处理控制流分成了多个控制流。联结与此相反，联结具有两个或者多个转移入口，只有一个出口。联结描述了不同的处理控制流合并到一起形成一个单向处理，如下图所示。

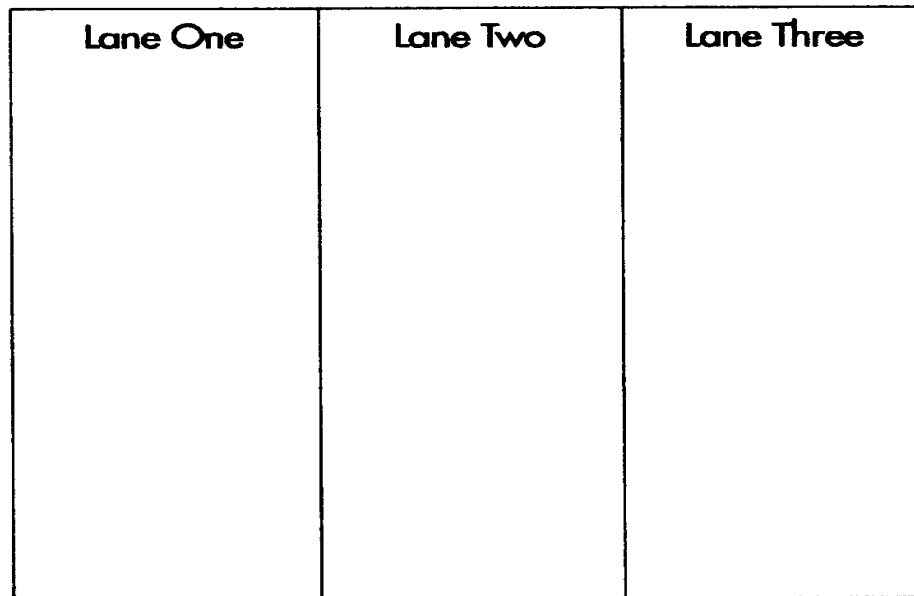


- 如果一个处理在其他处理之前到达了联结，它将会等待，直到所有的处理都准备好之后才会向联结传递控制权。
- 下例演示了分岔中的一个处理时间长于另一个的情况。由于我们不知道每一个活动有多长，因此不能保证哪一个首先完成。为此，我们在让用户访问应用程序之前插入了一个联结，以便确保两个独立的处理彼此连接在一起。

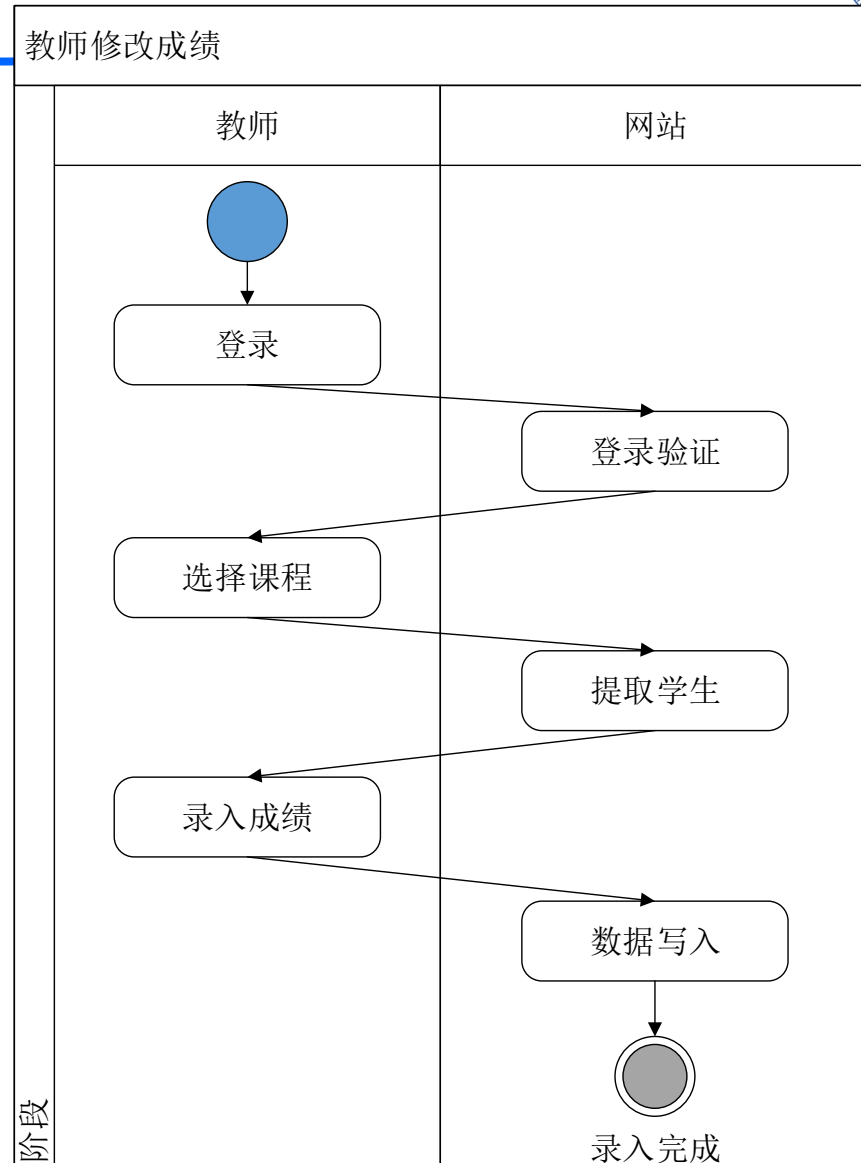


基本元素--泳道

- 泳道(swim-lane)进一步描述完成活动的对象，并聚合一组活动，因此泳道也是一种分组机制。
- 泳道可以使活动图非常整洁，因为它们在很大程度上增强了活动图的可读性。泳道使用几个大矩形框表示如下图所示。



- 在学校教务系统中教师和网站之间交叉转移控制流的活动图。
- 如果没有泳道，该活动图就无法说明教师使用了登录、选择课程、录入成绩活动，网站进行了验证登录、提取学生、写入成绩活动。



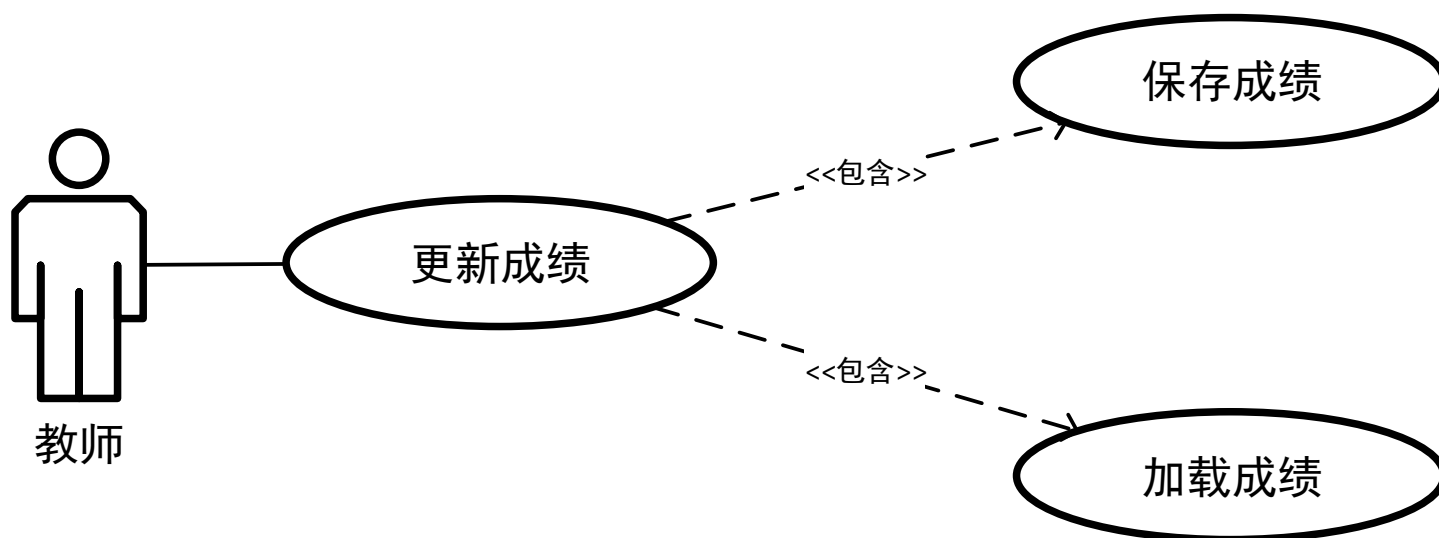
4.2 创建活动图

- 创建活动图共有5个任务：

- 1) 标识需要活动图的用例。
- 2) 建模每一个用例的**主路径**。
- 3) 建模每一个用例的**从路径**。
- 4) **添加泳道**来标识活动的事务分区。
- 5) 改进高层活动并**添加更多活动**到图中。

建模活动图--标识用例

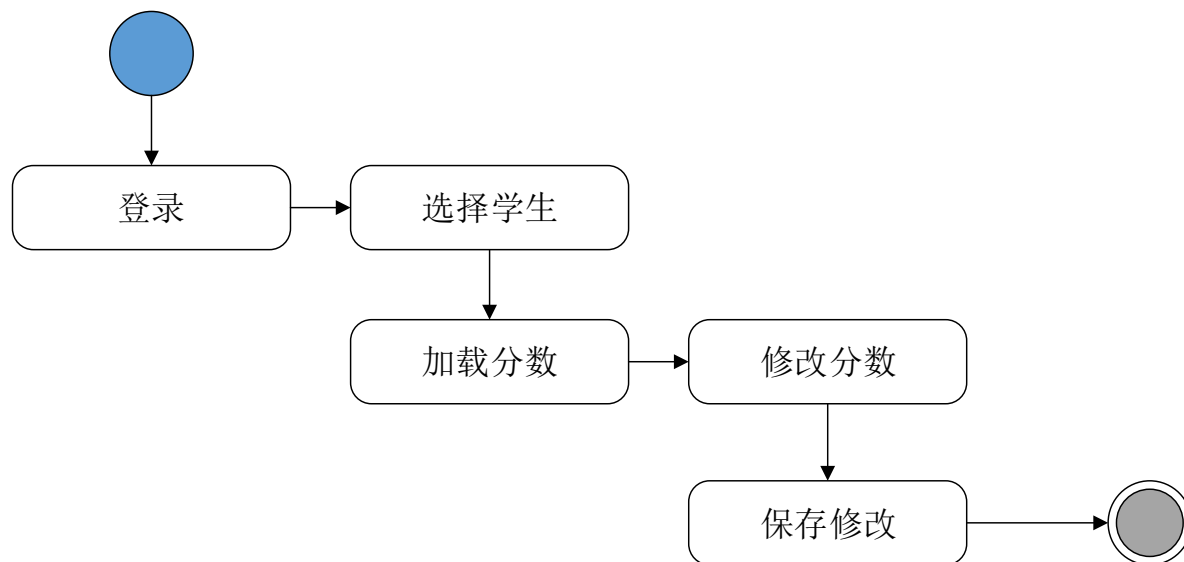
- 在建模活动图之前，需要首先确定要建模什么。下面的教师更新分数用例是一组较大用例的一部分，我们就从它开始。如下图所示。这个用例实际上使用了3个用例。我们不仅有更新成绩用例，还有保存成绩和加载成绩用例。



建模活动图--建模主路径

(1) 建模主路径

- 在开始创建用例的活动图时，往往先建立一条明显的路径执行工作流，然后从该路径进行扩展。**该路径仅考虑用例的正常活动路径**(登录、选择学生、加载他们的分数，修改分数，保存修改结果等活动过程)没有考虑任何错误和判断的路径。



建模活动图--添加泳道

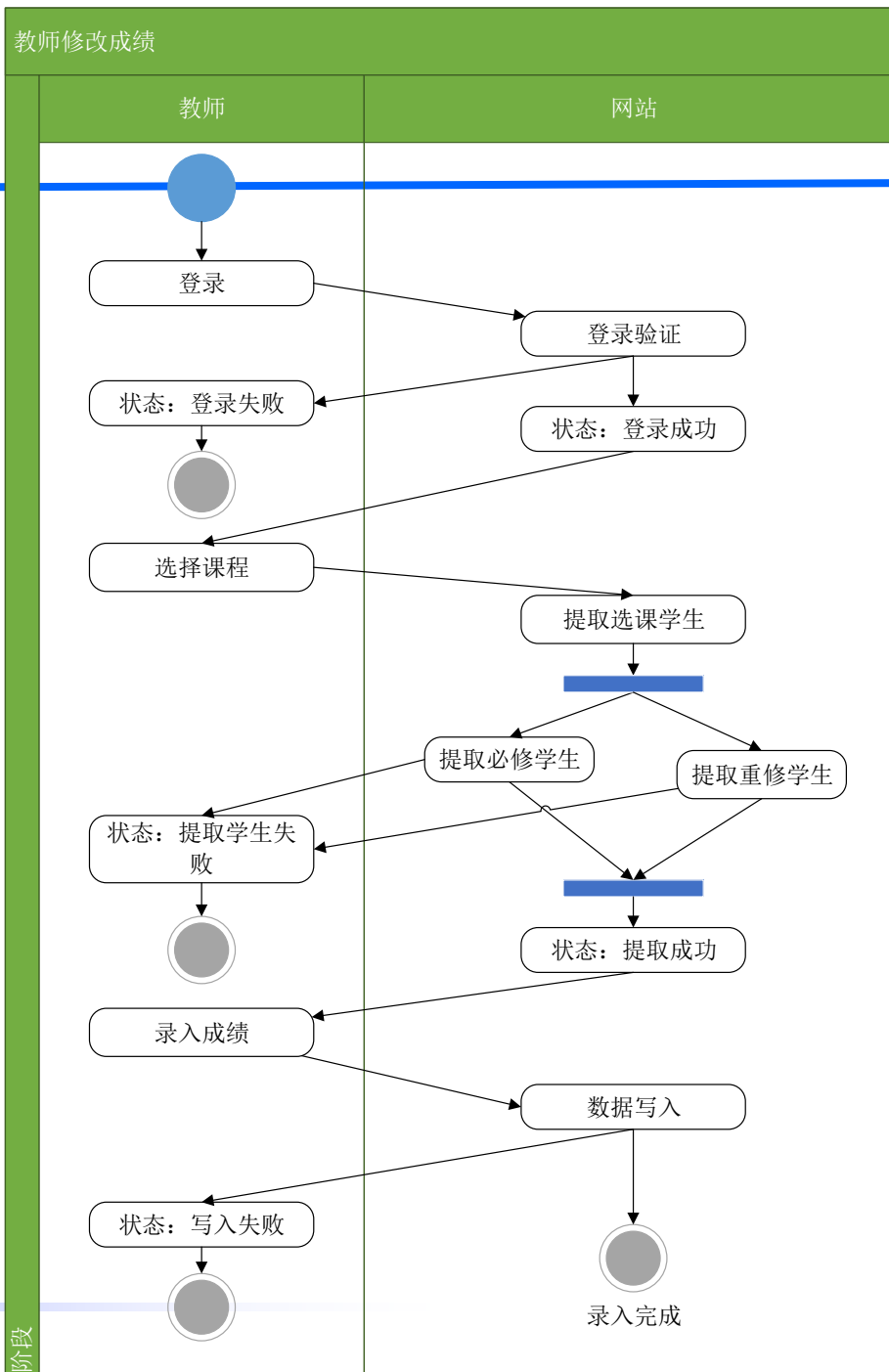
(2) 建模从路径。考虑用例其他可能的工作流情况。如处理错误，或许是执行其他活动。

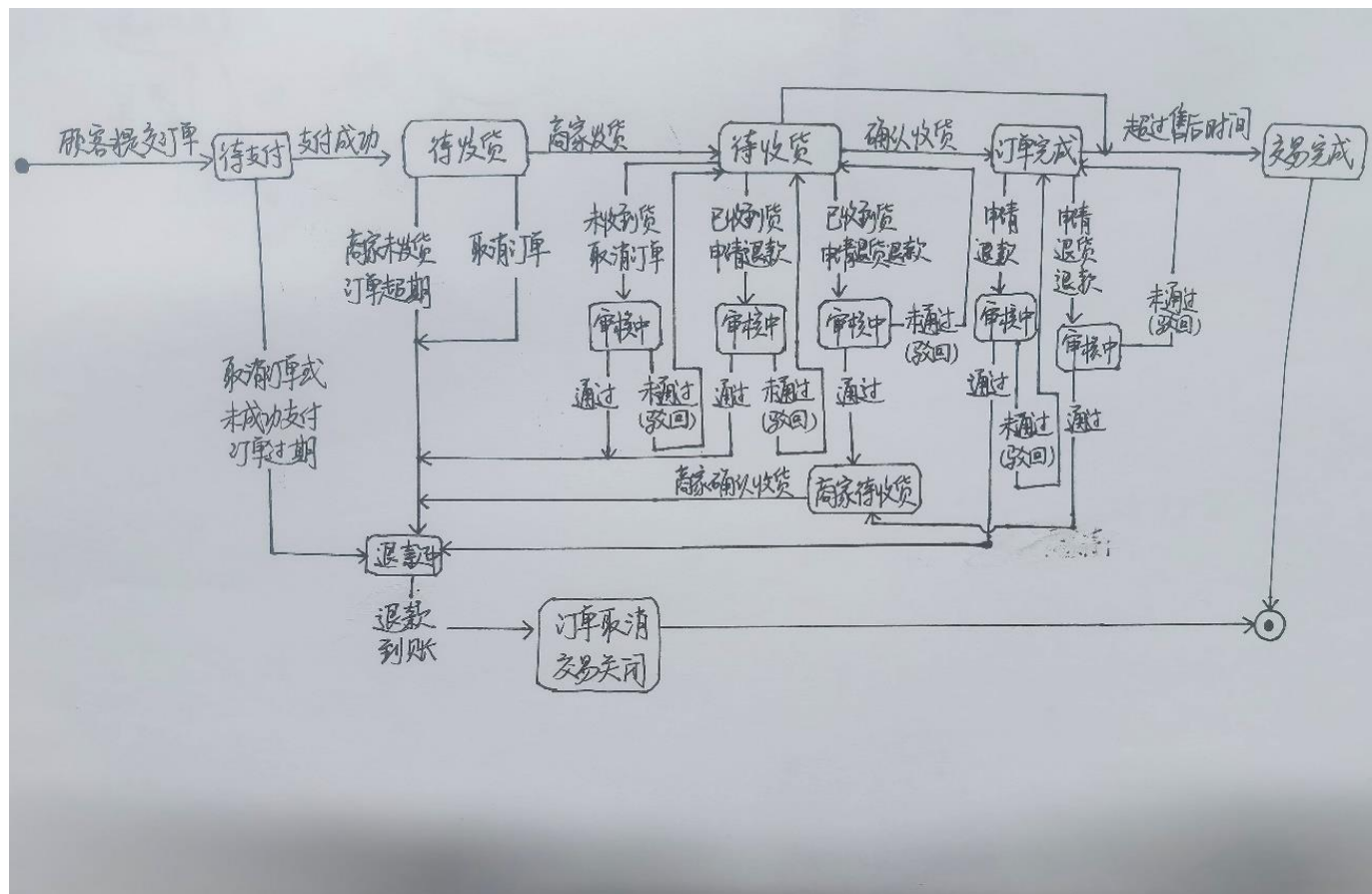
(3) 添加泳道

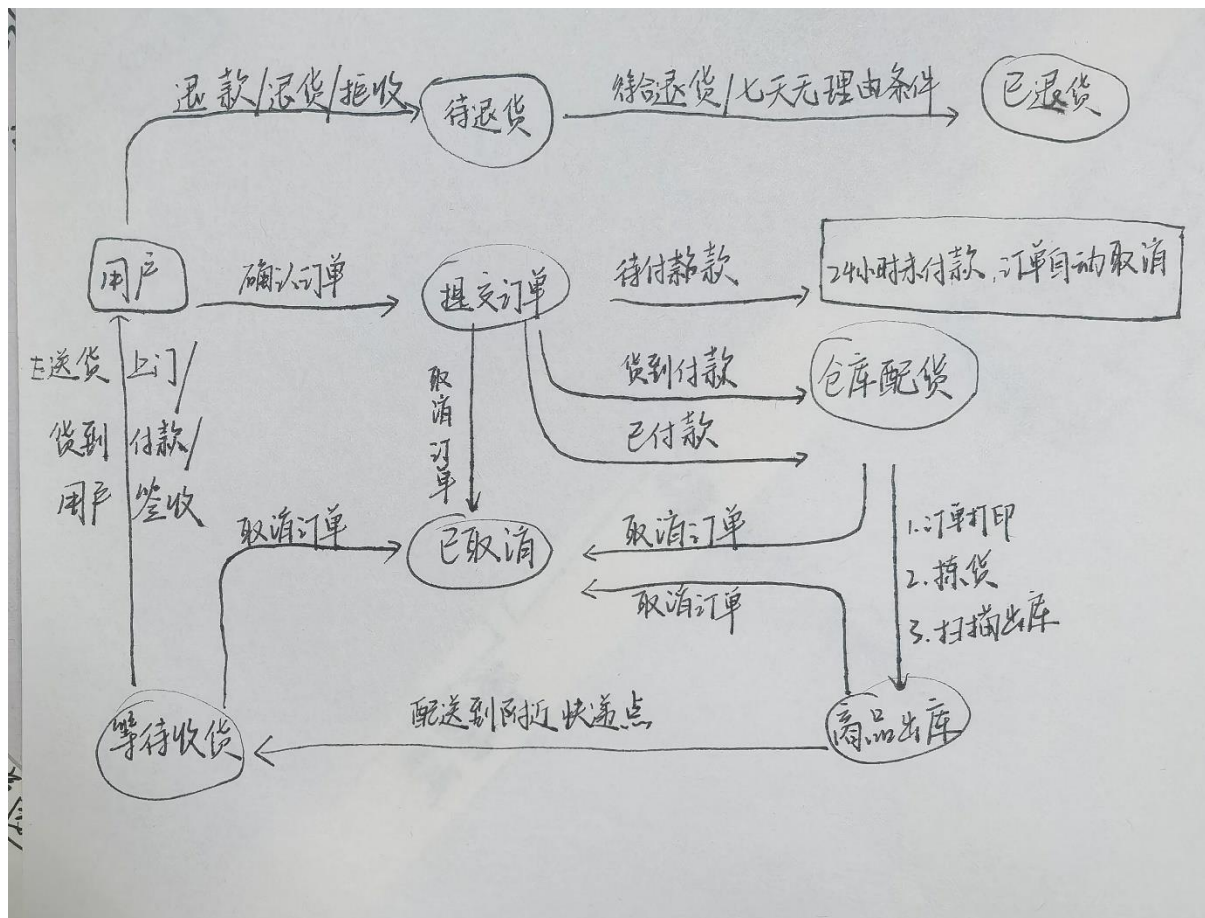
- 可把活动图分成了两个游泳道，第一个游泳道是教师，第二个是网站。教师是用例的参与者，而网站是提供后台功能的泛化组件。
- 这里，我们将再一次反复向活动图添加更多的细节。加载学生名单时需要同时加载第一次选课的学生和重新学生。同时我们还需要考虑可能的出错，如登录失败，在本例中，我们要添加状态以便指示现在处于哪一个转折点。
- 最后，我们决定让每一个出错消息都在活动图中的自己活动中。

建模活动图

教师修改成绩







状态:

S₀: 开始

S₁: 订单确认

S₂: 订单出库

S₃: 订单运送

S₄: 订单签收

S₅: 订单完成

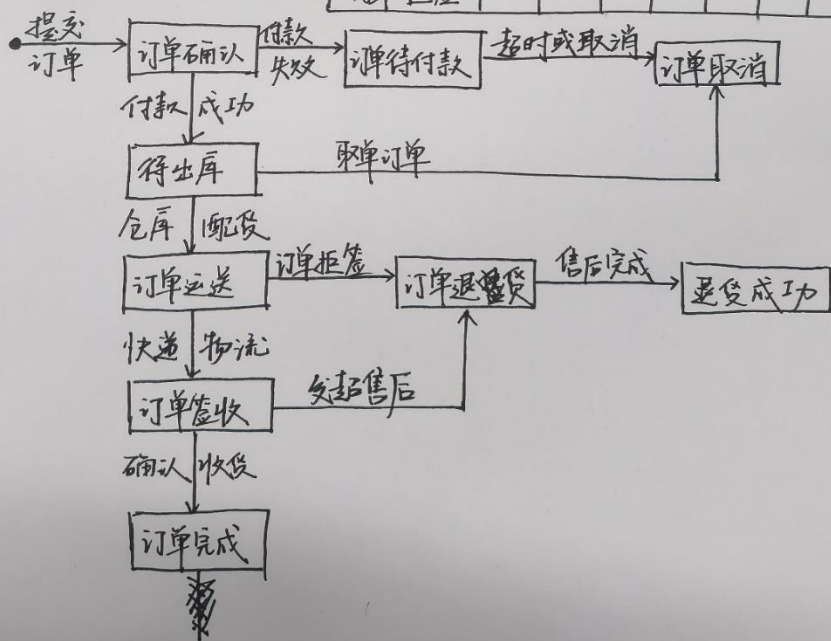
S₆: 订单待付款

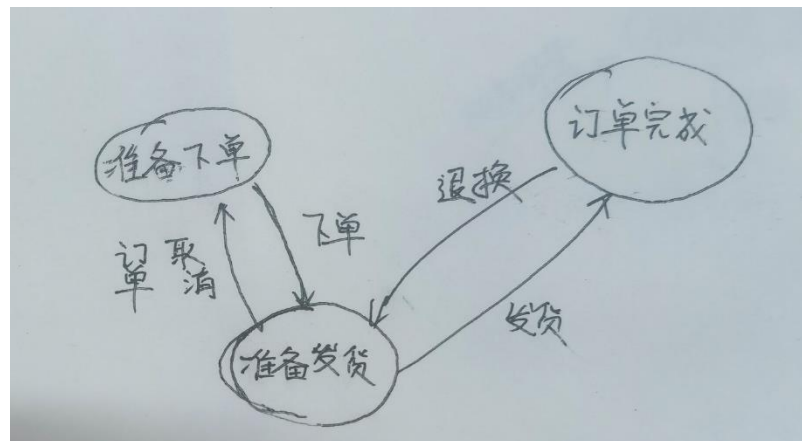
S₇: 订单取消

S₈: 订单退货

S₉: 退货成功

事件 \ 状态	S ₀	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇	S ₈	S ₉
提交订单	S ₁									
付款成功		S ₂								
仓库配货			S ₃							
快递运送				S ₄						
签收收货					S ₅					
付款失败		S ₆								
时间超时						S ₇				
发起售后					S ₈					
售后完成								S ₉		
订单拒签				S ₈						





5 UML状态图

- 状态是对象执行了一系列活动的结果。一个对象在其生存周期中可具有多个状态，通常一种状态具有时间的稳定性。
- 模型不必同时具有开始和结束状态，因为模型可以总是运行，从不停止。
- 状态图中可以包含0到多个开始状态。状态图中也可以包含多个结束状态，每一个都表示一个模型能够终止的点。



事件和转移

- UML预设了3个标准事件，而且都是无参数事件。Entry（进入）事件用于指明进入该状态时的特定动作；exit（退出）事件用于指明退出该状态时的特定动作；do（执行）事件用于指明在该状态中时执行的动作。

飞行状态

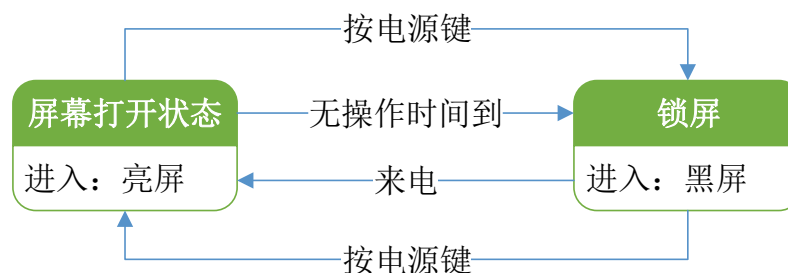
状态变量：时间
进入 / 记录起飞时间
执行 / 飞行里程
退出 / 记录降落时间

- 一个对象的状态改变称为状态转移，通常是由事件触发的，事件用来指示是什么导致了模型中状态的变化。

事件

在UML中，有4类事件。

- (1) 条件变为真，表示状态迁移条件满足。
- (2) 收到来自外部对象的信号(signal)，表示为状态迁移上的事件特征，也称为消息。
- (3) 收到来自外部对象的某个操作中的一个调用，对象用状态的转换而不是用固定的处理过程实现操作。
- (4) 状态迁移上的时间表达式。在状态图中，一般应标出触发转移的事件表达式。如果转移上未标明事件，则表示在源状态的内部活动执行完毕后自动触发转移。



条件、决策点和同步条

3. 条件

- 条件用来描述状态转移的前提。

4. 决策点

- 决策点为建模状态图时提供了方便，通过在中心位置分别转移到各自的方向，从而提高了状态图的可视性，

5. 状态图中使用同步条

- 同步条是为了说明并发工作流的分岔与联合。

例子

